GoKart

QField-Plugin zur Pflanzenerfassung

Detaillierte Anleitung: Einrichtung, Geländearbeit, Standortfotos, Abgleich und Fehlerbehebung



Plugin-Stand: v0.3 (qfield_plugin/) bzw. Basisversion (qfield/plugin/)

Zielplattform: QField 4.x auf Android und iPad; QField 3.x per Fallback (ungetestet)

Desktop: QGIS ≥ 3.22 mit Plugin „GoKart Desktop“ 0.4.0

Repository: github.com/Emu77/GoKart

Stand der Anleitung: 25. Oktober 2026

Erstellt auf Basis des Quellcodes im Repository (Commit 0d4fa51).

Inhalt

1 Überblick 3
2 Voraussetzungen 4
3 Datenmodell 4
4 Projekt am PC vorbereiten 5
5 Projekt aufs Gerät bringen 7
6 Arten im Gelände erfassen 8
7 Standortfotos (ab v0.3) 10
8 So funktioniert die Suche 10
9 Daten zurück an den PC 11
10 Fehlerbehebung 12

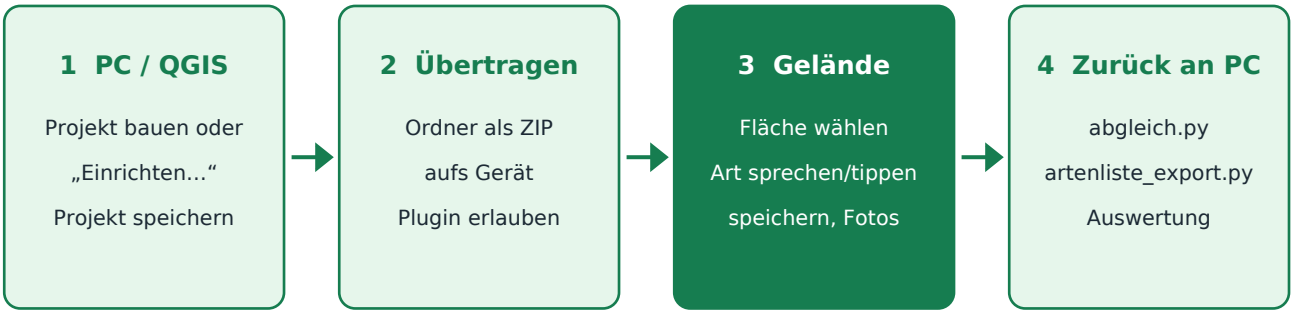
1 Überblick

GoKart ist ein Projekt-Plugin für **QField**, mit dem Pflanzenarten im Gelände per **Spracheingabe** oder Tastatur erfasst und direkt einer **Kartierfläche** (Biotop-, Vegetations- oder Teilfläche) zugeordnet werden. Der Kernablauf ist bewusst kurz gehalten:

Fläche wählen → Pflanzennamen sprechen oder tippen → aus max. 5 Vorschlägen wählen → speichern.

Die Suche ist fehlertolerant (Kölner Phonetik + Levenshtein), sodass auch Hör- und Diktierfehler wie „kwerkus robur“ zu *Quercus robur* führen. Unbekannte Schreibweisen kann man als eigenes Synonym speichern – beim nächsten Mal werden sie exakt erkannt. Ab Version 0.3 kommen Standortfotos mit Kommentar und GPS-Position hinzu.

Gesamtablauf



Bestandteile

Komponente	Ort im Repository	Aufgabe
QField-Plugin (Basis)	qfield/plugin/gokart.qml + gokart_suche.js	Arterfassung; feste Layernamen flaechen / artnachweise
QField-Plugin v0.3	qfield_plugin/GoKart_Wohnung.qml	wie Basis, plus Konfiguration über Projektvariablen und Standortfotos
Suchlogik	gokart_suche.js	Normalisierung, Phonetik, Gewichtung (identisch in allen Varianten)
GoKart Desktop	qgis_plugin/gokart_desktop/	QGIS-Plugin: Projekt einrichten, Arten am PC erfassen, Export
Werkzeuge	qfield/werkzeuge/	Projekt bauen, Plugin kopieren, Abgleich, Artenliste, Zusatztaxa
Tests	qfield/test/	Suchtest (Node) und QField-Simulation (PySide6)

Hinweis: Die QML-Datei des Plugins **muss genauso heißen wie das QGIS-Projekt** und daneben liegen: `gokart.qgs` → `gokart.qml`, `GoKart_Wohnung_qfield.qgs` → `GoKart_Wohnung_qfield.qml`. Nur dann erkennt QField es als Projekt-Plugin.

2 Voraussetzungen

Mobilgerät (Android oder iPad)

- **QField 4.x** (getestet). Für QField 3.x gibt es einen API-Fallback, der aber noch ungetestet ist.
- Eine Tastatur mit **Offline-Diktat**, da das Plugin die Spracheingabe der Bildschirmtastatur nutzt: z. B. **FUTO Keyboard** (FUTO Voice Input) oder **Gboard** mit heruntergeladenem deutschen Offline-Sprachpaket. Auf dem iPad die Diktierfunktion der Systemtastatur.
- GPS bzw. Standortfreigabe für QField, wenn Flächen per GPS gewählt werden sollen.
- Kamerafreigabe für QField, wenn Standortfotos (v0.3) genutzt werden.
- Das Plugin selbst braucht **kein Internet**; nur die OSM-Hintergrundkarte wird online geladen.

> **Tipp:** Der Ablauf wurde am 12.10.2026 auf einem Android-Smartphone (QField 4.x) mit 376 Biotopflächen erfolgreich getestet. Der Betrieb auf dem iPad ist Ziel, aber noch nicht dokumentiert getestet – beim ersten Einsatz bitte Abschnitt 5 und die Fehlerbehebung im Blick behalten.

PC (Ubuntu 24.04 oder Windows)

- **QGIS ≥ 3.22**, für die Werkzeuge unter Ubuntu zusätzlich `python3-qgis` und `qgis-providers`.
- Plugin **GoKart Desktop** (Ordner `qgis_plugin/gokart_desktop` in den QGIS-Pluginordner kopieren und aktivieren).
- Referenzdaten: `GoKart_Pflanzen.sqlite` (aus BfN-Skript 519) und optional `germansl.sqlite` (GermanSL 1.5) für zusätzliche deutsche Namen.
- Optional für Tests: Node.js und `pip install PySide6-Essentials`.

```
sudo apt install python3-qgis qgis-providers
```

3 Datenmodell

Alle Daten liegen in einem GeoPackage (`gokart.gpkg`) neben dem Projekt. Die Referenztabellen werden beim Öffnen des Dialogs einmal in einen Suchindex geladen.

Tabelle	Typ	Wichtige Felder	Bemerkung
flaechen	Polygon	flaeche_id (eindeutig), name, bemerkung	Kartierflächen; alternativ beliebiger Polygon-Layer (v0.3)
artnachweise	Tabelle	flaeche_id, taxon_id, wissenschaftlicher_name, deutscher_name, original_eingabe, erfasst_am, bemerkung	eine Art je Fläche nur einmal
eigene_synonyme	Tabelle	taxon_id, synonym, schluessel, angelegt_am	entsteht beim Erfassen
taxon	Tabelle	taxon_id, wissenschaftlicher_name, deutscher_name / anzeige_name, rang, autor_name	Referenz, schreibgeschützt (ca. 6.100 Taxa)
suchnamen	Tabelle	taxon_id, name, art, schluessel	Referenz, ca. 15.600 Suchnamen
fotos	Tabelle	flaeche_id, taxon_id, datei, kommentar, aufgenommen_am, lon, lat	nur v0.3; wird von abgleich.py angelegt

Zwischen Fläche und Artnachweisen besteht eine **1:n-Beziehung**. Dadurch zeigt das Flächenformular in QField den Reiter „Arten“ mit allen Nachweisen der Fläche.

Koordinatensystem

Das Build-Skript übernimmt das KBS der Flächen-Quelldatei. Ohne eigene Flächen ist der Standard **EPSG:25833**; das Projekt `gokart_qfield` nutzt **EPSG:25832** (UTM 32N, Norddeutschland). GPS-Positionen (EPSG:4326) und die Kartenmitte werden vor der Flächenabfrage automatisch in das KBS des Flächenlayers transformiert.

> **Achtung:** Berlin/Brandenburg liegt in UTM-Zone 33. Bei Flächen dort prüfen, ob die Quelldatei wirklich 25832 hat.

4 Projekt am PC vorbereiten

Es gibt zwei Wege: das Projekt komplett mit dem Build-Skript erzeugen (Weg A) oder ein vorhandenes QGIS-Projekt mit dem Desktop-Plugin einrichten (Weg B, nur mit Plugin v0.3).

Weg A – Projekt mit build_gokart_projekt.py bauen

Das Skript erzeugt GeoPackage und QGIS-Projekt (inklusive Relation und Auswahllisten) und kopiert das Plugin aus `qfield/plugin/` dazu.

```
cd GoKart/qfield/werkzeuge
python3 build_gokart_projekt.py \
  --db /media/emu/daten/gpb/GoKart/pflanzen/GoKart_Pflanzen_Datenbank/GoKart_Pflanzen.sqlite \
  --germansl "/media/emu/daten/gpb/GoKart/pflanzen/germansl/GermanSL 1.5/germansl.sqlite" \
  --ziel /media/emu/daten/gpb/GoKart/qfield_projekt
```

Eigene Kartierflächen übernehmen – zuerst die Felder der Quelle anzeigen lassen, dann mit ID-Feld bauen:

```
# 1. Felder anzeigen
python3 build_gokart_projekt.py --db … --ziel … --flaechen /pfad/biotope.shp
# 2. mit ID-Feld (und optional Namensfeld) bauen
python3 build_gokart_projekt.py --db … --germansl … --ziel … \
  --flaechen /pfad/biotope.shp --id-feld BIOTOP_NR --name-feld BT_TEXT --neu
```

Option	Bedeutung
--db	GoKart_Pflanzen.sqlite (Pflicht)
--ziel	Zielordner des QField-Projekts (Pflicht)
--germansl	germansl.sqlite für zusätzliche deutsche Namen (ca. 500 ergänzt)
--zusatz / --zusatztaxa	eigene CSV-Dateien (Standard: zusatznamen.csv, zusatztaxa.csv)
--flaechen	eigene Polygone (GPKG, SHP …) statt Testflächen
--flaechen-layer	Layername bei GeoPackages mit mehreren Layern
--id-feld / --name-feld	Feld mit eindeutiger Flächen-ID / optionale Bezeichnung
--id-auto PRÄFIX	fortlaufende IDs (z. B. K6-001, K6-002 …) – Reihenfolge wie in der Quelldatei
--neu	vorhandenes gokart.gpkg überschreiben (vorher automatische .bak-Sicherung)

> **Achtung:** Mit `--neu` gehen alle Erfassungen im alten GeoPackage verloren. Vorher die Daten vom Gerät zurückholen und abgleichen! Doppelte IDs erhalten ein Suffix (-2, -3), Flächen ohne ID heißen OHNE-ID-<fid> – beides meldet das Skript. Mit --id-auto können sich Nummern bei einer geänderten Quelldatei verschieben; besser ein dauerhaftes ID-Feld pflegen.

Weg B – Vorhandenes Projekt mit „GoKart Desktop“ einrichten

Plugin v0.3 liest seine Konfiguration aus Projektvariablen. Damit lässt sich jeder Polygon-Layer mit beliebigem Schlüsselfeld verwenden.

1. Projekt in QGIS öffnen. Das GoKart-GeoPackage (mit taxon, suchnamen, eigene_synonyme, artnachweise) muss als Layer eingebunden sein.
2. Menü **Erweiterungen → GoKart: Projekt einrichten…** oder im GoKart-Panel **Einrichten…**.
3. **Flächen**: Layer, **Schlüsselfeld** (eindeutige ID) und optional **Anzeigefeld** (z. B. Raum, name, Bezeichnung) wählen.
4. **Artnachweise**: Layer und Feld **Verweis auf Fläche** (Standard: flaeche_id) wählen.
5. Die Prüfung im Dialog beachten: Anzahl Flächen, Flächen ohne Schlüssel, doppelte Schlüssel, unpassende Feldtypen (Text ↔ Zahl).
6. Haken **QGIS-Beziehung anlegen/aktualisieren** gesetzt lassen – so bekommt QField den Reiter „Arten“. Konkurrierende Beziehungen auf dasselbe Feld bietet der Dialog zum Entfernen an.
7. OK und das **Projekt speichern (Strg+S)** – erst dann stehen die Variablen in der .qgs-Datei und QField erhält sie.

Projektvariable	Inhalt
gokart_flaechen_layer	Layer-ID der Flächen
gokart_flaechen_key	Schlüsselfeld der Flächen
gokart_flaechen_label	Anzeigefeld (optional)
gokart_nachweis_layer	Layer-ID der Artnachweise
gokart_nachweis_fk	Fremdschlüssel in den Artnachweisen
gokart_relation	ID der angelegten Beziehung (gokart_flaeche_nachweise)

Fehlen die Variablen, sucht das Plugin nach Layern namens „Wohnung – Teilflächen“ bzw. „flaechen“ und „artnachweise“ mit dem Feld flaeche_id (Fallback). Welche Quelle verwendet wurde, steht in der Diagnosezeile.

5 Projekt aufs Gerät bringen

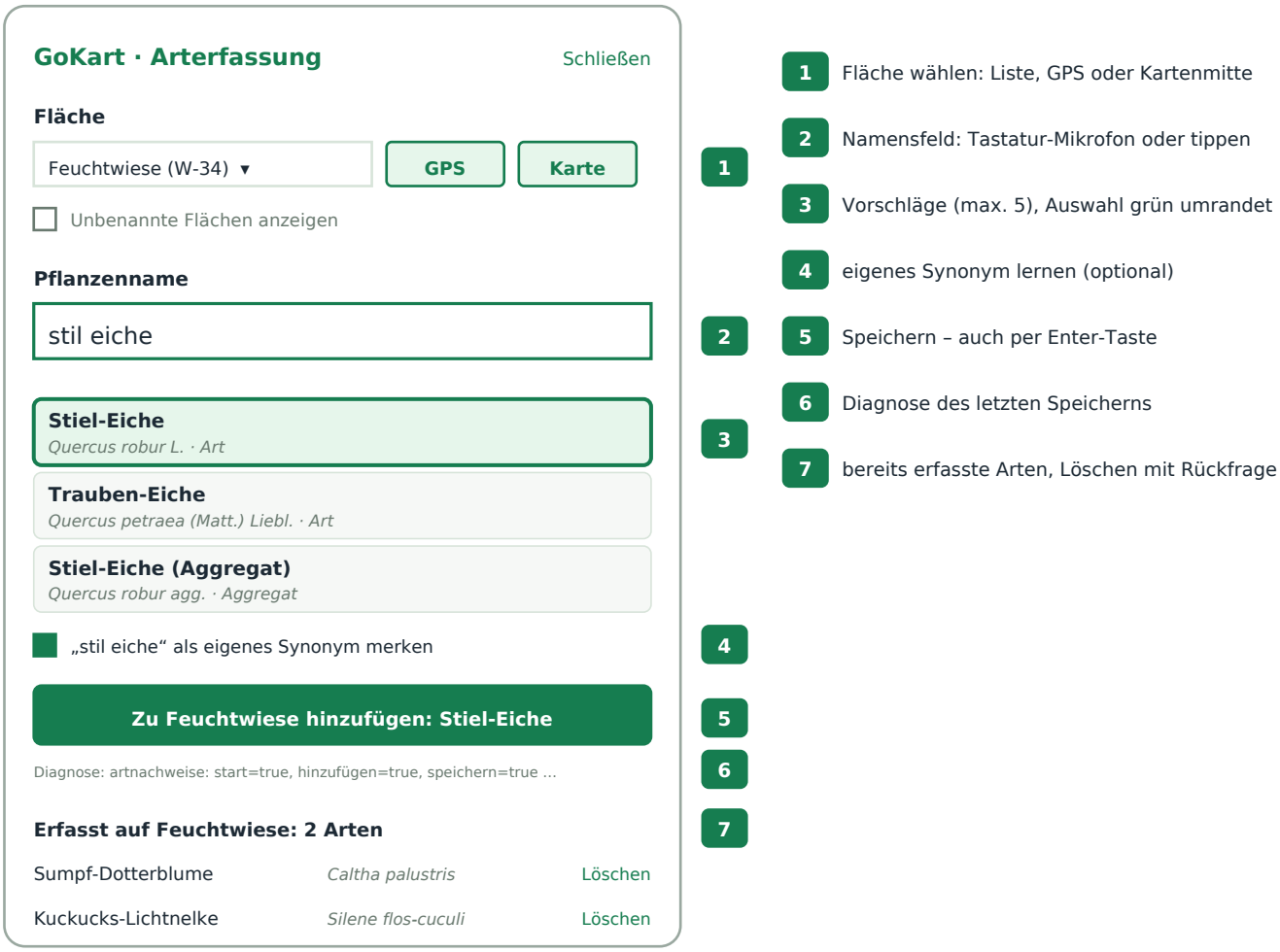
1. Den Projektordner **inklusive** `.gpkg`, `.qgs`, Plugin-`.qml`, `gokart_suche.js` und den SVG-Icons als ZIP packen.
2. ZIP aufs Gerät kopieren und in QField über **Projekt aus Ordner/ZIP importieren** öffnen – oder den Ordner direkt in den QField-Projektordner kopieren. Auf dem iPad z. B. über die Dateien-App in den QField-Bereich.
3. Das Projekt (.qgs) öffnen. QField fragt, ob das **Projekt-Plugin** geladen werden darf → **zulassen** (optional „immer“).
4. In der Plugin-Leiste erscheinen grüne Rundknöpfe: **Blatt** = Arterfassung, ab v0.3 zusätzlich **Kamera** = Standortfotos.

Achtung: `gokart.gpkg` ist absichtlich nicht im Git-Repository (enthält eigene Flächen und Erfassungen). Nach einem frischen Klonen öffnet sich das Projekt deshalb ohne Layer – das GeoPackage muss lokal vorliegen.

Hinweis: Beim ersten Öffnen des Dialogs werden die Pflanzendaten geladen („Pflanzendaten werden geladen …“). Danach meldet ein Hinweis z. B. „GoKart: 6101 Taxa geladen (… ms)“. Der Index bleibt bis zum Schließen des Projekts im Speicher.

6 Arten im Gelände erfassen

Den grünen **Blatt-Knopf** in der Plugin-Leiste antippen. Es öffnet sich der Dialog „GoKart · Arterfassung“:



Schritt 1 – Fläche wählen

- **Liste**: Fläche aus dem Auswahlfeld wählen. Angezeigt wird „Name (ID)“, sortiert nach Name. Gibt es nur eine Fläche, ist sie automatisch aktiv.
- **GPS**: die Fläche, in der die aktuelle GPS-Position liegt. Voraussetzung: gültige Position in QField.
- **Karte**: die Fläche unter der Kartenmitte – praktisch im Emulator oder wenn das GPS ungenau ist.
- Überlappen sich Flächen, wählt v0.3 die **kleinste** Fläche am Punkt (z. B. Teilfläche statt Gesamtgebiet).
- v0.3: Flächen ohne Eintrag im Anzeigefeld sind ausgeblendet; mit **Unbenannte Flächen anzeigen** erscheinen sie wieder.

Schritt 2 – Pflanzennamen eingeben

- Ins Feld **Pflanzenname** tippen und das **Mikrofon der Tastatur** nutzen – oder normal tippen.
- Deutsche und wissenschaftliche Namen, botanische Synonyme und eigene Synonyme werden gefunden.

- Die Suche startet automatisch 250 ms nach der letzten Änderung und zeigt höchstens **5 Vorschläge**.
- Unter jedem Vorschlag stehen voller wissenschaftlicher Name mit Autor, Rang und ggf. „Synonym: …“ bzw. „eigenes Synonym“.
- „Keine passende Art gefunden.“ erscheint in Rot, wenn nichts passt.

Schritt 3 – Vorschlag wählen

Gibt es genau **einen exakten Treffer**, ist er bereits ausgewählt (grün umrandet). Sonst den passenden Vorschlag antippen. Bei 42 doppelt vorkommenden wissenschaftlichen Namen (s. l. / s. str., z. B. *Urtica dioica*) hilft der volle Name mit Autor bei der Unterscheidung.

Schritt 4 – Eigenes Synonym merken (optional)

War die Eingabe kein bekannter Name (z. B. „Stil Eiche“), erscheint die Option **„… als eigenes Synonym merken“**. Mit Haken wird die Eingabe beim Speichern in `eigene_synonyme` abgelegt und ab sofort exakt erkannt – eigene Synonyme haben in der Suche die höchste Priorität.

Schritt 5 – Speichern

- Großen grünen Knopf **„Zu <Fläche> hinzufügen: <Art>“** antippen – oder einfach **Enter** drücken, wenn eine Art ausgewählt ist. Ablauf beim Diktieren also: sprechen → Enter.
- Gespeichert werden Fläche, taxon_id, wissenschaftlicher und deutscher Name, die **Originaleingabe** und der Zeitpunkt.
- Die Änderung wird sofort dauerhaft ins GeoPackage geschrieben (startEditing → hinzufügen → commitChanges).
- Ist die Art auf dieser Fläche schon erfasst, kommt die Meldung „… ist auf … bereits erfasst.“ – es entsteht kein Duplikat.
- Danach wird das Feld geleert und ist sofort für die nächste Art bereit.

Erfasste Arten ansehen und löschen

Unten im Dialog steht „Erfasst auf <Fläche>: n Arten“ mit der Liste (neueste oben). **Löschen** antippen → Knopf wird zu **„Sicher?“** → innerhalb von 3 Sekunden erneut antippen. Die Nachweise sind außerdem im Flächenformular von QField im Reiter „Arten“ sichtbar.

Diagnosezeile

Nach jedem Speichern zeigt die kleine Zeile „Diagnose: …“ die einzelnen Schritte (start, hinzufügen, speichern, Bearbeitungsmodus). Bei Fehlern ist sie rot und enthält die Fehlermeldung des Layers. Hängende ungespeicherte Änderungen werden beim Öffnen des Dialogs automatisch nachgespeichert.

7 Standortfotos (ab v0.3)

Über den zweiten grünen Knopf (**Kamera**) öffnet sich „GoKart · Standortfotos“.

1. Fläche (Standort) wählen – wie bei der Arterfassung per Liste, GPS oder Karte. Die aktive Fläche ist in beiden Dialogen dieselbe.
2. **„Foto aufnehmen – <Fläche>“** antippen. Die Kamera-App öffnet sich.
3. Nach der Aufnahme wird das Bild unter `fotos/gokart_<Fläche>_<Datum_Uhrzeit>.jpg` im Projektordner gespeichert, auf max. **1600 px** (längste Seite) verkleinert und mit Zeitpunkt und – falls verfügbar – GPS-Position eingetragen.
4. Unter dem Foto einen **Kommentar** tippen oder diktieren (z. B. Licht, Feuchte) und **Kommentar speichern** (oder Enter).
5. **Foto entfernen** (mit „Sicher?“-Rückfrage) löscht nur den Tabelleneintrag – die Bilddatei bleibt im Ordner fotos/.

Achtung: Die Tabelle `fotos` wird nicht vom Plugin angelegt. Meldet QField „Fototabelle fehlt“, am PC einmal `abgleich.py … --ausfuehren` laufen lassen und das neue ZIP auf dem Gerät importieren.

8 So funktioniert die Suche

1. **Normalisieren**: Kleinschreibung, ä→a, ß→ss, Leer- und Bindestriche entfernen – „Stiel Eiche“ = „Stiel-Eiche“.
2. **Stufen**: exakt → Wortanfang → enthalten.
3. **Kölner Phonetik** gegen Hör- und Erkennungsfehler („kwerkus robur“ → *Quercus robur*, „Spitzwegerisch“ → Spitz-Wegerich).
4. **Levenshtein-Distanz** bei Tippfehlern.
5. **Gewichtung**: eigenes Synonym > deutscher Name > wissenschaftlicher Name > botanisches Synonym; bei Gleichstand Art vor Unterart vor Aggregat vor Gattung.

Eine Suche dauert am PC etwa 30 ms. Der Suchtest mit Echtdaten (32 Fälle) läuft vollständig durch.

Namensbestand erweitern

- `zusatznamen.csv` – zusätzliche gebräuchliche Namen (Format `wissenschaftlicher_name;name`, z. B. *Trifolium pratense*;Rot-Klee). Wird beim Projektbau übernommen.
- `zusatztaxa.csv` – fehlende Taxa, z. B. Moose. **taxon_id ab 90001 fest vergeben und nie ändern**, weitere Suchnamen mit | trennen.
- In ein bestehendes GeoPackage ohne Neubau einspielen (Erfassungen bleiben, Sicherung wird angelegt, beliebig oft ausführbar):

```
python3 qfield/werkzeuge/zusatztaxa_einspielen.py <projekt.gpkg> [zusatztaxa.csv]
```

9 Daten zurück an den PC

Abgleich PC ↔ Tablet (abgleich.py)

Gleicht Artnachweise, Fotos und eigene Synonyme zwischen PC-Projekt und Tablet-Export ab. QGIS und QField vorher schließen.

```
# nur Unterschiede anzeigen, nichts ändern
python3 qfield/werkzeuge/abgleich.py <PC-Projekt .qgs|Ordner> <Tablet-Export .zip|Ordner>
# Unterschiede auf BEIDEN Seiten übernehmen
python3 qfield/werkzeuge/abgleich.py <PC> <Tablet> --ausfuehren
```

- PC-Projekt wird direkt aktualisiert (vorher Sicherung *.bak).
- Fürs Tablet entsteht ein neues ZIP `<Tablet>_abgleich_<Datum>.zip`; das Original bleibt unberührt. Das aktuelle Plugin aus qfield_plugin/ kommt automatisch mit (abschalten: --ohne-plugin).
- Ein Nachweis ist eindeutig durch Fläche + taxon_id, ein Foto durch seinen Dateinamen. Fehlende Einträge werden ergänzt (Vereinigung).
- Bemerkungen/Kommentare werden nur ergänzt, nie überschrieben; Konflikte werden gemeldet.
- **Gelöschte Einträge** erkennt das Skript nicht – sie kämen von der anderen Seite zurück. Ausschließen mit `--weglassen W-34:68005` (Fläche:taxon_id) bzw. `--foto-weglassen fotos/<Datei>.jpg` (mehrfach möglich).

Artenliste je Fläche (artenliste_export.py)

```
python3 qfield/werkzeuge/artenliste_export.py ~/Dokumente/GoKart_Wohnung_2026-10-23.zip
python3 qfield/werkzeuge/artenliste_export.py <Projekt> \
    -o ~/Gutachten/artenliste -t "Artenliste je Fläche"
```

Erzeugt eine druckbare **HTML**-Datei (im Browser als PDF speicherbar) und eine **CSV**. Liest nur, verändert nichts, braucht kein QGIS. Layer und Felder kommen aus den GoKart-Projektvariablen, sonst gelten die Standardnamen. Alternativ im Desktop-Plugin: Export als CSV oder als Artenliste (HTML).

Plugin in vorhandenen Projekten aktualisieren

`qfield/plugin/` ist die einzige Quelle des Basis-Plugins; Projektordner enthalten nur Kopien. Nie in der Kopie bearbeiten. GeoPackage und .qgs bleiben beim Aktualisieren unverändert:

```
# kopieren
python3 qfield/werkzeuge/plugin_aktualisieren.py gokart_qfield
# nur vergleichen (Exit-Code 1 bei Abweichung)
python3 qfield/werkzeuge/plugin_aktualisieren.py gokart_qfield --pruefen
```

10 Fehlerbehebung

Symptom / Meldung	Ursache	Lösung
Kein GoKart-Knopf in der Plugin-Leiste	QML-Name passt nicht zur .qgs, Plugin-Laden abgelehnt, oder Dateien fehlen	Namen angleichen (projekt.qgs ↔ projekt.qml); gokart_suche.js und Icons daneben legen; Projekt neu öffnen und Plugin zulassen
„QField-API nicht gefunden – Plugin inaktiv“	QField-Version zu alt bzw. API geändert	QField auf 4.x aktualisieren
„Layer taxon/suchnamen fehlen im Projekt“	GeoPackage nicht eingebunden oder fehlt	gokart.gpkg in den Projektordner, Layer in QGIS prüfen
Diagnose: „Flächen=FEHLT“ / „Nachweise=FEHLT“	Projektvariablen fehlen, Layer umbenannt	Am PC „Einrichten…“ ausführen und Projekt speichern; neu übertragen
„Keine gültige GPS-Position“	kein Fix oder Standort nicht freigegeben	Positionierung in QField aktivieren, warten; ersatzweise „Karte“ oder Liste
„Keine Fläche per GPS gefunden“	Position außerhalb aller Flächen oder falsches KBS	KBS der Flächen prüfen (25832/25833); Fläche aus der Liste wählen
Fläche nicht in der Liste	kein Schlüsselwert oder (v0.3) kein Anzeigename	Schlüssel pflegen; „Unbenannte Flächen anzeigen“ aktivieren
„Speichern fehlgeschlagen – Details unten im Dialog“	Layer schreibgeschützt, Datei gesperrt, Feldtyp unpassend	Diagnosezeile lesen; Typ Schlüssel ↔ Fremdschlüssel prüfen (Text ↔ Zahl)
„… ist auf … bereits erfasst“	gleiche Art schon auf dieser Fläche	kein Fehler – Duplikatschutz
Diktat erkennt botanische Namen schlecht	Spracherkennung der Tastatur	deutschen Namen sprechen; Fehlhörungen als eigenes Synonym speichern
„Fototabelle fehlt …“	Tabelle fotos noch nicht angelegt	abgleich.py --ausfuehren, neues ZIP importieren
„Bilddatei nicht gefunden“	Ordner fotos/ nicht mitkopiert	Projekt mit fotos/ übertragen bzw. abgleichen
Gelöschte Art taucht nach Abgleich wieder auf	Abgleich ist eine Vereinigung	--weglassen FLÄCHE:TAXON_ID verwenden
Dialog bei eingeblendeter Tastatur zu eng	kleines Display (bekannter offener Punkt)	Gerät quer halten oder Tablet nutzen

Tests am PC

```
python3 qfield/test/testdaten_export.py qfield_projekt/gokart.gpkg testdaten.json
node qfield/test/test_suche.js testdaten.json
QT_QPA_PLATFORM=offscreen python3 qfield/test/qml_harness.py qfield_projekt_kopie
```

Achtung: `qml_harness.py` simuliert QField und schreibt in das GeoPackage – nur mit einer **Kopie** ausführen.

Offene Punkte (laut Projektstand)

- Fallback für QField 3.x ungetestet; Betrieb auf dem iPad noch zu bestätigen.
- Dialoghöhe bei eingeblendeter Bildschirmtastatur auf kleinen Displays.
- Optional eigene Offline-Erkennung mit Vosk, falls das Tastatur-Diktat bei botanischen Namen nicht reicht (im Desktop-Plugin bereits optional vorhanden).

Kurzreferenz Geländeablauf

#	Aktion
1	Projekt öffnen, Plugin zulassen
2	Blatt-Knopf → Fläche per GPS / Karte / Liste
3	Ins Namensfeld tippen → Tastatur-Mikrofon → Namen sprechen
4	Vorschlag prüfen (ggf. antippen) → Enter
5	Bei Fehlhörung: Haken „als eigenes Synonym merken“
6	Nächste Art – Feld ist schon leer und aktiv
7	Kamera-Knopf → Foto → Kommentar diktieren → speichern
8	Zu Hause: Export holen → abgleich.py → artenliste_export.py